

Les Tuples en Python

💡 Définition et Création

Un tuple (ou pultet) est une collection de données **ordonnée** mais **immuable** (non modifiable). Une fois créé, on ne peut ni ajouter, ni supprimer, ni modifier ses éléments. Il est reconnaissable par les parenthèses () qui l'entourent.

```
root@python:~$  
  
# Création d'un tuple vide  
mon_tuple = ()  
# Création d'un tuple avec des éléments  
point = (10, 20)  
# Un tuple peut contenir des types différents  
personne = ("Bob", 18, True)  
# Obtenir le nombre d'éléments avec len()  
taille = len(personne)  
print(f"Le tuple contient {taille} éléments.")  
# Affiche "Le tuple contient 3 éléments."
```

⚠ **Attention :** Pour créer un tuple avec **un seul** élément, il faut obligatoirement mettre une virgule : `mon_tuple = (5,)`

📖 Parcourir un tuple

On utilise la boucle for exactement comme avec une liste.

```
root@python:~$  
  
# Parcours par element  
notes = (12, 15, 18)  
for n in notes:  
    print(n)
```

```
Affiche :  
12  
15  
18
```

```
root@python:~$  
  
# Parcours par index  
notes = (12, 15, 18)  
for i in range(len(notes)):  
    print(f"Note n°{i} : {notes[i]}")
```

```
Affiche :  
Note n°0 : 12  
Note n°2 : 15  
Note n°2 : 18
```

```
root@python:~$  
  
# Parcours mixte  
notes = (12, 15, 18)  
for i, n in enumerate(notes):  
    print(f"Note n°{i} : {n}")
```

```
Affiche :  
Note n°0 : 12  
Note n°2 : 15  
Note n°2 : 18
```

📍 Indexation et Accès aux éléments

Comme pour les listes et les chaînes, chaque élément possède un index.

⚠ **L'indexation commence à 0.**

```
root@python:~$  
  
couleurs = ("rouge", "vert", "bleu", "jaune")  
# Accès par index positif  
print(couleurs[0]) # Affiche "rouge"  
# Accès par index négatif  
print(couleurs[-1]) # Affiche "jaune"  
# Le slicing : Extraire une partie [début:fin]  
print(couleurs[1:3]) # Affiche ('vert', 'bleu')
```

😞 Immuabilité

⚠ Contrairement aux listes, le tuple est **immuable**. On ne peut pas modifier son contenu après sa création. C'est une structure sécurisée pour des données qui ne doivent pas changer (ex: coordonnées GPS, constantes).

```
root@python:~$  
  
triplet = (1, 2, 3)  
# Cette instruction générera une erreur !  
# triplet[0] = 10  
# TypeError: 'tuple' object does not support item assignment
```

👤 Vérifier la présence d'une valeur

On utilise l'opérateur in pour tester si un élément appartient au tuple.

```
root@python:~$  
  
langages = ("Python", "C", "Java")  
if "Python" in langages:  
    print("Le langage est présent.")  
# Affiche "Le langage est présent."  
if "HTML" not in langages:  
    print("Le langage n'est pas présent.")  
# Affiche "Le langage n'est pas présent."
```

⚙ Affectation Multiple (Unpacking)

Le tuple permet d'affecter plusieurs variables d'un coup. C'est une fonctionnalité très utilisée en Python, notamment pour retourner plusieurs valeurs avec une fonction.

```
root@python:~$  
  
# Affectation multiple  
coordonnees = (48.85, 2.35)  
latitude, longitude = coordonnees  
print(latitude) # 48.85
```

Échanger deux variables sans variable temporaire

```
a = 5  
b = 10  
a, b = b, a  
print(a, b) # Affiche 10 5
```

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM



FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM